NightShift: Batch LLM Inference on Charging Smartphones

Anonymous Author(s)

ABSTRACT

Billions of smartphones spend every night plugged in, on Wi-Fi, and idle. We ask whether that pool can run inference for a large language model too big for any single phone, and what it would take. A 70B-parameter model at 4-bit needs about 40 GB, so one copy is a 16-stage pipeline of phones that leave whenever their owners unplug them. Using a simulator whose phone and network parameters are explicit, replaceable assumptions, we find three things. First, the pool is a night-time resource: 98% of a day's tokens are produced between 22:00 and 08:00. Second, per-token latency is seconds (2.2 s at 60 ms per hop), so the useful workload is batch work, not chat. Third, the dominant cost of churn is not lost computation but moving model shards: a Petals-style design re-downloads 345.6 GB of weights per million tokens. NightShift fixes each phone's shard for good, forms pipelines from phones expected to leave together, and consolidates broken pipelines at dawn. With 1,000 phones it cuts shard traffic 15× at equal or slightly higher throughput; its advantage grows with the pool, to 96× less traffic and +19% throughput at 4,000 phones. Keeping warm standbys, the obvious fix, cuts throughput by 39%. We release the simulator and a benchmark kit so the assumed parameters can be replaced with measurements.

1 INTRODUCTION

Volunteer computing has a long record: BOINC and SETI@home turned idle desktops into a scientific computer [2], and Google's federated learning system trains models on phones only while they are "idle, charging and on an unmetered network" [3]. Large language models (LLMs) are the largest compute consumer of the decade, which invites the question: could the idle phones of the world run them?

The question has an easy wrong answer in each direction. "Yes, a billion phones beat any data centre" ignores that generation is sequential and that a large model does not fit on a phone. "No, phones are too small" ignores that the same was said of volunteer desktops, and that systems such as Petals [4, 5] already serve 100B-class models from pipelines of volunteer GPUs over the internet. The interesting question is quantitative: *for which workloads, at what throughput and cost, and which design choices matter?*

This paper makes three contributions.

(1) A first-principles analysis (§2) of what a phone pool can and cannot do: it suits throughput-oriented batch jobs, not interactive chat, and it pays more for memory traffic and energy per token than a GPU.
(2) A churn-aware simulator (§5) driven by a diurnal model of when phones are charging and on Wi-Fi. Every phone, network and user parameter is explicit and can be replaced by a measurement; we release a benchmark kit that produces those measurements on Android phones.
(3) NightShift (§4), a placement and recovery design built on one observation from the simulator: once phones come and go, moving model shards costs more than redoing computation. Sticky shard ownership, lifetime-grouped pipelines and dawn consolidation cut shard downloads 6–96× across every setting we tried, and the gain grows with pool size. Dedicated standbys, the standard remedy for preemption [21], make things worse.

We are explicit about what is and is not measured. All results here come from simulation with stated assumptions (Table 1); §7 describes the measurements that will replace them.

2 THE PHYSICS OF A PHONE POOL

Memory. A 70B model quantised to about 4 bits per weight occupies roughly 40 GB; we model 80 layers of 0.5 GB. An Android app running in the background can rarely hold more than a fraction of a phone's RAM; we assume 45%, so an 8 GB phone offers 3.6 GB. With the key-value (KV) cache for a batch of 32 sequences of 768 tokens, a five-layer block needs 3.0 GB, so a model copy is a pipeline of 16 phones, and phones with 6 GB of RAM cannot take part at all (90% of our pool is eligible).

Latency. Generating one token visits all 16 stages in order. At roughly 15 ms per layer on a mid-range phone and 60 ms per wide-area hop, a token takes about 1.2 s of compute plus 1.0 s of network, i.e. around two seconds. Neither speculative decoding [12] nor faster phones change the order of magnitude while the hops remain. A conversational assistant is therefore out of reach over the internet; a household network (5 ms hops) brings latency to 1.3 s.

Throughput. Latency need not limit throughput. With many sequences in flight, a pipeline keeps every stage busy, as in GPipe-style micro-batching [9] and continuous batching servers [10, 25]. Batch workloads such as offline evaluation, synthetic data generation, document summarisation or embedding a corpus tolerate hours of latency and are a large share of LLM demand, the same regime FlexGen targets on a single GPU [20].

Memory traffic and energy. The price is efficiency. A data-centre GPU serving a batch of 128 reads each weight once per 128 tokens, about 0.3 GB of weight traffic per token for a 70B model. A phone pipeline with 32 sequences over 16 stages processes micro-batches of two, so it reads about 20 GB per token, roughly 60× more. At an assumed 4 W per active phone and 1 W while waiting, the simulated pool spends about 6 J per token; we have not measured a GPU server on the same workload, and that comparison is part of the validation plan. The phone pool's argument is therefore not efficiency but *marginal cost*: the hardware is already bought, already powered at night and otherwise idle. Whether that argument survives battery wear and the owner's electricity bill is a question for measurement and incentive design, which we leave open (§8).

3 RELATED WORK

Volunteer and federated computing. BOINC [2] established redundant execution and work units for untrusted volunteers but ran

independent jobs. Federated learning [3, 15] runs on phones at production scale and established the eligibility rule we adopt (charging, Wi-Fi, idle); each phone trains a full small model, and nothing is split across devices. Device availability in federated learning has been measured at large scale [11, 23]; those traces are natural inputs to our simulator.

LLMs over unreliable consumer hardware. Petals [4, 5] serves large models from pipelines of volunteer GPUs and recovers from departures by routing around them and replaying the lost stage's cache; it is the baseline we call *replay*. SWARM [18], Learning-@home [19] and DiLoCo [6] train over unreliable or poorly connected nodes. SpotServe [16] serves LLMs on preemptible cloud instances and Bamboo [21] adds redundancy to survive preemption. Those nodes are GPUs with tens of gigabytes and minutes of warning; ours are phones with a few gigabytes and no warning.

Splitting models across nearby devices. LinguaLinked [27] runs one LLM across several Android phones, Galaxy [24] and EdgeShard [26] split transformers across edge devices, and exo [7] does so for a household's laptops and phones; earlier, MoDNN [14] and DeepThings [28] split CNNs. These assume a small, stable set of devices. We study the opposite regime: a large pool in which membership changes all the time.

Single-phone inference. LLM in a flash [1], PowerInfer-2 [22] and runtimes such as llama.cpp [8] make single phones faster, and MobileLLM [13] designs models that fit one phone. These are complementary: a faster phone shortens every stage, and a model that fits one phone needs no pipeline at all, which is the right choice whenever such a model is good enough.

4 NIGHTSHIFT

A coordinator holds a job queue and a fixed *shard catalogue*: the model cut into 16 blocks of 5 layers. A phone joins the pool when it is charging, on Wi-Fi and idle, and leaves without warning. A pipeline is one phone per block; it advances a batch of 32 sequences in lock step. When a member leaves, its sequences pause until the stage is refilled. NightShift has three parts.

Sticky shards. Every phone is assigned a *home block* once and keeps it in flash storage indefinitely. A pipeline stage is only ever filled by a phone whose home block matches; loading from flash takes seconds, whereas downloading 2.5 GB over a residential link takes minutes and costs the owner data. If no matching phone is idle, the vacant stage waits up to five minutes before falling back to a phone that must download the block. Because the catalogue is fixed, the coordinator only needs to keep the 16 home-block populations balanced.

Lifetime-grouped pipelines. A pipeline lives only as long as its shortest-lived member. The coordinator predicts when each phone's charging session ends, from that user's history of sessions starting at a similar time of day, sorts each block's idle phones by predicted end, and builds the i-th pipeline from the i-th phone of every block. Phones expected to stay until morning end up together, and short daytime sessions are grouped with each other instead of cutting long-lived pipelines short.

Table 1: Simulator parameters. All are assumptions until replaced by measurements from the benchmark kit (§7).

Model	80 layers × 0.5 GB, KV 4 KiB/token/layer
Workload	batches of 32 seq., 512 prompt + 256 output tokens
Prefill cost	1/8 of a decode token
Phones	RAM, share of pool, ms per layer: 6 GB, 10%, 30; 8 GB, 40%, 18; 12 GB, 35%, 12; 16 GB, 15%, 10 (log-normal spread σ=0.2)
Usable RAM	45% of physical
Network	60 ms per hop; downlink median 50 Mbit/s
Flash read	1 GB/s
Power	4 W computing, 1 W waiting
Night charging	85% of nights, start 23:00±1 h, 7±1.5 h
Day charging	1.5 sessions/day, median 35 min, 60% on Wi-Fi

Dawn consolidation. In the morning, phones leave faster than idle phones arrive, leaving many pipelines with one or two vacant stages that nobody can fill. Waiting wastes every phone in those pipelines. When at least two pipelines are broken, NightShift dissolves the most damaged one (most vacancies, then least progress) and lends its phones, which already hold blocks of the same catalogue, to the others. A pool that shrinks keeps as many *complete* pipelines as it can afford.

Recovery. A replacement phone recomputes the KV cache of its block for the tokens processed so far (prefill is cheaper per token than decode), as in Petals; we compare this with restarting the batch.

What we chose not to do. Keeping k warm standbys per stage turns a failure into a fast swap but takes $k \times 16$ phones out of service per pipeline. We evaluate it as a baseline.

5 SIMULATION METHODOLOGY

The simulator advances in 10-second ticks over 24 hours from noon, after seven days of history used only to train the session-end predictor. Each phone has a RAM class, a per-layer decode time and a downlink speed drawn from Table 1, and a sequence of charging sessions drawn from the diurnal model, or read from a real trace. A pipeline's per-token latency is the sum of its stage times plus one hop per stage; its throughput is limited by $\max(\text{latency}, 16 \cdot \max_i \text{stage}_i)$ per round, since 16 micro-batches are in flight. A micro-batch of b tokens costs $1+0.1(b-1)$ times a single token. We assume steady state: every phone already holds one block in flash from an earlier night, so day-one onboarding traffic is excluded for all policies alike. Results are means over 5 seeds with 1,000 phones; error bars show one standard deviation.

We compare seven policies that differ only in placement and recovery: *restart* and *replay* (random placement, Petals-style replay), *standby* (replay plus one standby per stage), and the three NightShift components alone (*lifetime*, *consolidate*, *sticky*) and together (*NightShift*). The upper bound places every available, eligible phone in a complete pipeline instantly.

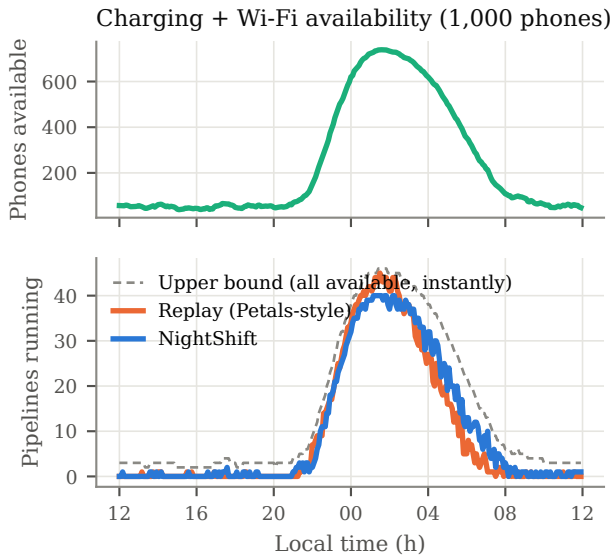


Figure 1: One simulated day. Top: phones available. Bottom: pipelines running and an upper bound that assumes instant, perfect placement.

6 RESULTS

The pool is a night-time machine. Availability swings from 39 to 739 of 1,000 phones over the day (Figure 1), and 98% of tokens are produced between 22:00 and 08:00. Daytime sessions are short and scattered, so few complete 16-phone pipelines form. A phone pool is best treated as a batch system that drains a queue overnight.

Moving shards, not lost work, is the cost of churn. Figure 2 compares the policies. Restart and replay differ by only -5% in throughput, so the computation lost to departures is small. What separates the policies is data movement: under replay, every repair recruits whichever phone is idle, which rarely holds the needed block, and the pool downloads 345.6 GB of weights for every million tokens it produces, enough to make a volunteer's monthly data cap the binding constraint.

The components only work together. Each part of NightShift alone is a poor trade. Sticky shards halve traffic (173.5 GB) but lose -8% throughput, because a vacant stage now waits for a phone that holds its block. Lifetime grouping alone is worse than random (106 tokens/s, 549.5 GB), because ranking phones by predicted stay overrides the reuse of cached blocks. Consolidation alone gives the highest throughput (+8%) but keeps traffic near replay's (314.9 GB). Combined, consolidation supplies the home-block phones that sticky repairs wait for: NightShift downloads 23.7 GB per million tokens (15× less than replay), suffers 46% fewer stage failures, and delivers 116 tokens/s per 1,000 phones against 112 for replay (+4%). Its stages spend 86% of their time computing rather than waiting, against 66%. If shard traffic were free, consolidation alone would be the better policy at this pool size; it is not free for the phones' owners.

Standbys are a bad trade on phones. One standby per stage cuts throughput by 39%: it removes half of the pool from service to protect pipelines whose expected lifetime, at night, is hours. Standbys pay off when failures are frequent relative to the cost of idling capacity, which is the case for preemptible cloud GPUs [21] but not here.

NightShift gets better with scale. With more phones, each block has more idle owners to choose from, so a sticky repair almost never has to wait or download (Figure 3). From 500 to 4,000 phones, NightShift's throughput rises from 108 to 133 tokens/s per 1,000 phones and its traffic falls from 59.7 to 3.9 GB per million tokens, while replay stays flat (114 to 112) and its traffic grows. At 500 phones, NightShift is -5% behind replay in throughput; at 4,000 it is +19% ahead with 96× less traffic. This is the regime that matters: a deployment worth building has far more than 4,000 phones.

Sensitivity. Figure 4 varies one assumption at a time. Three matter a lot. The share of nights a phone charges moves NightShift's throughput from 61 to 141 tokens/s per 1,000 phones between 50% and 100% of nights (at 50%, replay is slightly ahead). The batch size moves it from 64 (16 sequences) to 158 (48), because more sequences keep more stages busy; memory for the KV cache caps this. The relative cost of prefill moves it from 61 to 146, since every repair replays the prompt. Longer daytime sessions help only modestly (113 to 131), confirming that the night carries the system. Hop latency sets per-token latency (Figure 5: 1.3 s at 5 ms, 2.2 s at 60 ms, 3.2 s at 120 ms) and, because a round cannot be shorter than one token's trip, also throughput (132 to 81). Grouping pipelines by region, so that hops stay short, is therefore worth more than any scheduling refinement. Across every setting, NightShift downloads 6–96× less than replay.

7 TOWARD VALIDATION

Simulation with assumed parameters can show which design choices matter, but not absolute numbers. We release a benchmark kit that replaces each assumption with a measurement and re-runs every experiment unchanged:

- **Compute.** llama.cpp [8] on Android (Termux) measures decode and prefill speed for an 8B model, the cost of batching 1–8 sequences, sustained speed under thermal throttling, and battery draw. Per-layer time is scaled to the 70B layer by bytes of weights read, since phone decode is memory-bandwidth bound.
- **Network.** Round-trip time and throughput between phones on the same and on different networks.
- **Pipeline.** llama.cpp's RPC backend runs an 8B model split over 1–4 phones, validating the latency model (sum of stage times plus hops).
- **Availability.** A logger records charging and Wi-Fi state every five minutes for 14 days on volunteers' phones; the trace replaces the synthetic diurnal model. Large published traces [23] are an alternative.

Measured values: [per-device table: decode tok/s, prefill ratio, batching cost, sustained tok/s, W; RTT; RPC latency vs. prediction; trace statistics].

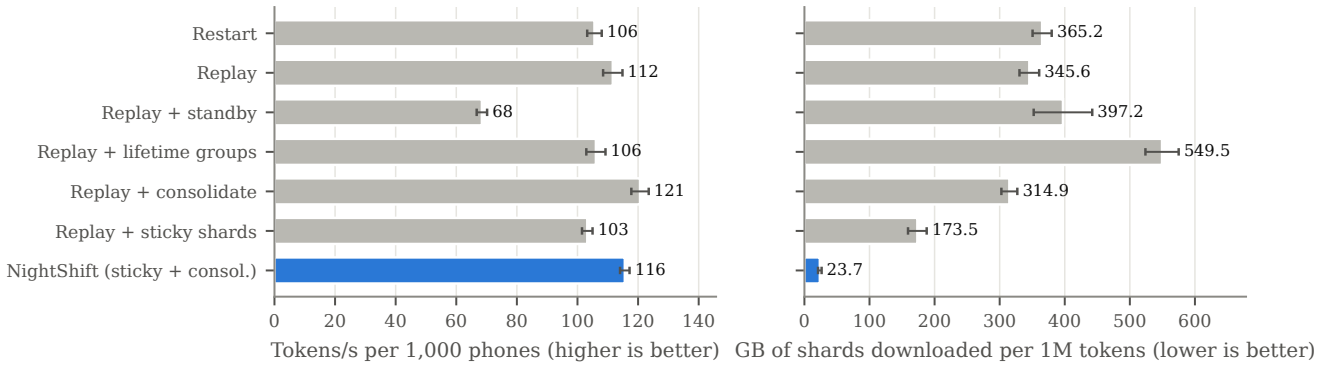


Figure 2: Throughput and shard traffic of the seven policies (1,000 phones, mean ± s.d. over 5 seeds).

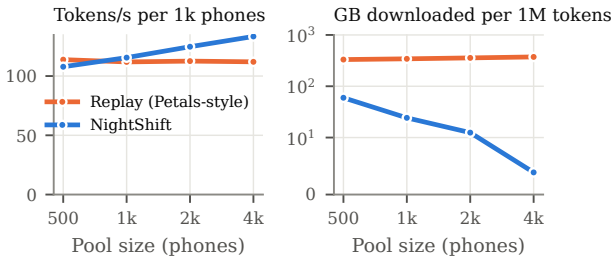


Figure 3: NightShift improves with pool size; replay does not. Shard traffic is on a logarithmic scale.

8 LIMITATIONS AND ETHICS

Privacy. A stage sees the activations of the prompts that pass through it, and activations can leak their input. We therefore restrict the design to work whose inputs are not sensitive (public corpora, synthetic data) until a defence exists.

Integrity. A volunteer can return wrong activations. Redundant execution [2] doubles cost; lightweight checks such as TOPLOC [17] are promising and untested on phones.

Owners. Overnight compute adds heat and battery wear and uses the owner's electricity. Participation must be explicit, revocable and limited by temperature; the federated learning eligibility rule is a minimum, not a licence. We have not modelled incentives.

Model fidelity. The simulator ignores stragglers within a stage, thermal throttling over a night, Wi-Fi contention in one home, and iOS's stricter background limits. Shard downloads are counted, not simulated as competing traffic.

9 CONCLUSION

A pool of charging phones can, in principle, run a model far larger than any phone, but only as an overnight batch system with seconds of latency per token and worse energy efficiency than a GPU. Within that regime, the decisive design choice is to keep model shards where they are: sticky ownership, pipelines of phones that leave together and consolidation at dawn remove most shard traffic without losing throughput, and the benefit grows with the pool, while standbys hurt. The next step is to replace our assumptions with measurements from real phones and real owners.

REFERENCES

[1] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL).*

[2] David P. Anderson. 2004. BOINC: A System for Public-Resource Computing and Storage. In *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing (GRID).* 4–10.

[3] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. 2019. Towards Federated Learning at Scale: System Design. In *Proceedings of Machine Learning and Systems (MLSys).* arXiv:1902.01046.

[4] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative Inference and Fine-tuning of Large Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (System Demonstrations).*

[5] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin Raffel. 2023. Distributed Inference and Fine-tuning of Large Language Models Over The Internet. In *Advances in Neural Information Processing Systems (NeurIPS).*

[6] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc'Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. 2023. DiLoCo: Distributed Low-Communication Training of Language Models. arXiv:2311.08105.

[7] exo labs. 2024. exo: Run your own AI cluster at home with everyday devices. https://github.com/exo-explore/exo.

[8] Georgi Gerganov and contributors. 2023. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp.

[9] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Advances in Neural Information Processing Systems (NeurIPS).*

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP).*

[11] Fan Lai, Yinwei Dai, Sanjay S. Singapuram, Jiachen Liu, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury. 2022. FedScale: Benchmarking Model and System Performance of Federated Learning at Scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML).*

[12] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML).*

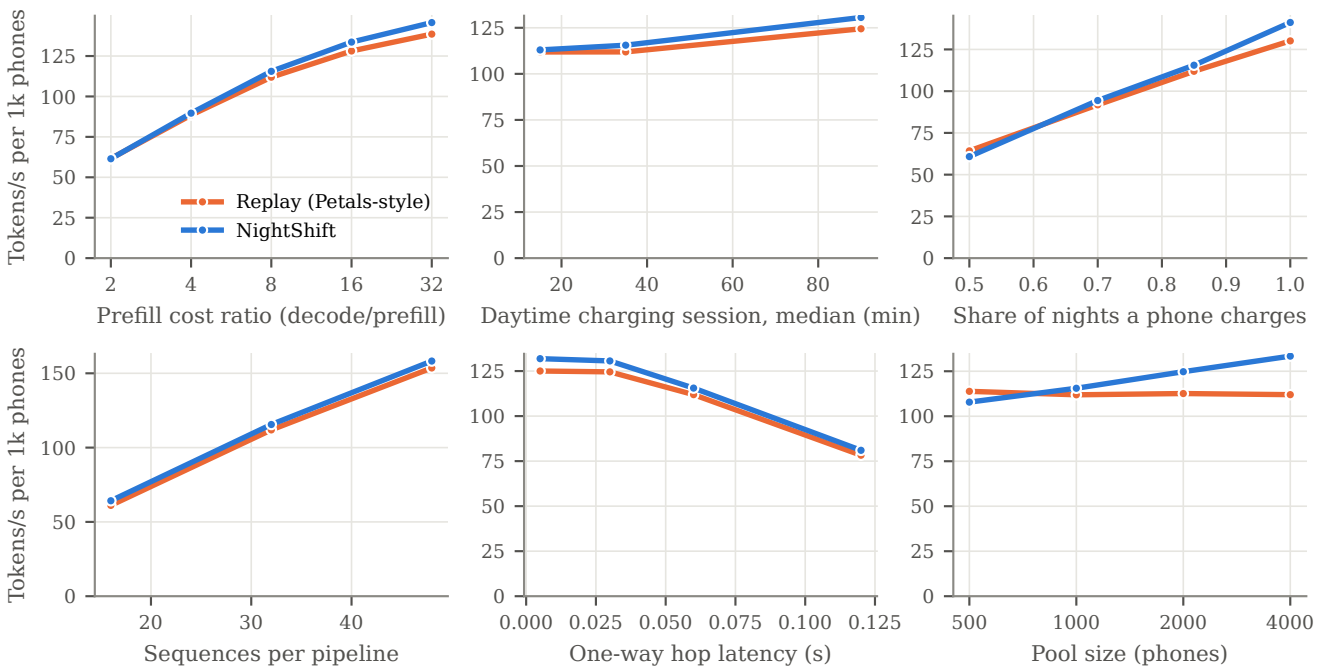


Figure 4: Sensitivity of throughput to the main assumptions, replay vs. NightShift (1,000 phones, mean of 3 seeds).

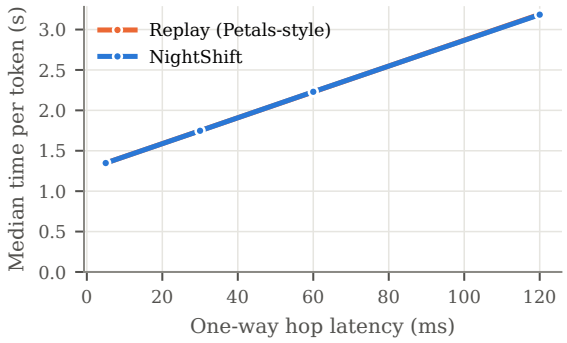


Figure 5: Median time per output token against one-way hop latency.

[13] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

[14] Jiachen Mao, Xiang Chen, Kent W. Nixon, Christopher Krieger, and Yiran Chen. 2017. MoDNN: Local Distributed Mobile Computing System for Deep Neural Network. In *Design, Automation & Test in Europe Conference (DATE)*.

[15] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*.

[16] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. SpotServe: Serving Generative Large Language Models on Preemptible Instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[17] Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. 2025. TOPLOC: A Locality Sensitive Hashing Scheme for Trustless Verifiable Inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 47196–47211.

[18] Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. 2023. SWARM Parallelism: Training Large Models Can Be Surprisingly Communication-Efficient. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

[19] Max Ryabinin and Anton Gusev. 2020. Towards Crowdsourced Training of Large Neural Networks using Decentralized Mixture-of-Experts. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[20] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

[21] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[22] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. arXiv:2406.06282.

[23] Chengxu Yang, Qipeng Wang, Mengwei Xu, Zhenpeng Chen, Kaigui Bian, Yunxin Liu, and Xuanzhe Liu. 2021. Characterizing Impacts of Heterogeneity in Federated Learning upon Large-Scale Smartphone Data. In *Proceedings of the Web Conference (WWW)*.

[24] Shengyuan Ye, Jiangsu Du, Liekang Zeng, Wenzhong Ou, Xiaowen Chu, Yutong Lu, and Xu Chen. 2024. Galaxy: A Resource-Efficient Collaborative Edge AI System for In-situ Transformer Inference. In *Proceedings of IEEE INFOCOM*. arXiv:2405.17245.

[25] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[26] Mingjin Zhang, Jiannong Cao, Xiaoming Shen, and Zeyang Cui. 2024. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. arXiv:2405.14371.

[27] Junchen Zhao, Yurun Song, Simeng Liu, Ian G. Harris, and Sangeetha Abdu Jyothi. 2023. LinguaLinked: A Distributed Large Language Model Inference

System for Mobile Devices. arXiv:2312.00388.

[28] Zhuoran Zhao, Kamyar Mirzazad Barijough, and Andreas Gerstlauer. 2018. DeepThings: Distributed Adaptive Deep Learning Inference on Resource-Constrained IoT Edge Clusters. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018).